# Ders 17 Genişletilmiş Notu: Vize 1 Sorularının Çözümü (Test ve Yazılı Bölüm)

## Ders 17 Genişletilmiş Notu: Vize 1 Sorularının Çözümü (Test ve Yazılı Bölüm)

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

### İçindekiler


- 1. Önce Ne İstendiğini Anlamak
- 2. Test Bölümü: On Sorunun Çözümü
- 3. Yazılı Soru 1: Onaltılık Karakter Dizisinin Onluk Karşılığı
- 4. Yazılı Soru 2: Matristen Sütun Silme


### 1. Önce Ne İstendiğini Anlamak

Bu ders bir uygulama dersidir ve birinci ara sınavın soruları sınıfla birlikte çözülür. Hoca sorulara geçmeden önce, bir problemi çözmeye başlamadan yapılması gereken ilk işin üzerinde durur: istenen şeyi tam olarak anlamak. Algoritma geliştirmekten de önce gelen adım budur.

Bunu bir benzetmeyle anlatır. Bir müşterinin ne istediğini tam olarak anlayamazsanız, onun sorununa onun istediği gibi bir çözüm üretemezsiniz. Bir problemle karşılaşınca hemen algoritmaya ve kod yazmaya odaklanırsanız, günün sonunda istenenden farklı bir şey teslim etmiş olursunuz.

> **Hocanın vurgusu:** Önce ne istendiğini anlamak zorundayız. Açıklamaları dikkatle okumadan yalnız algoritmaya ve koda odaklanmak yanlış sonuca götürür.

### 2. Test Bölümü: On Sorunun Çözümü

Hoca test sorularını ekranda ve tahtada, sınıfa sorarak tek tek çözer. Soruların hepsinin derste gösterilen bilgilerin yorumlanmasıyla çözülebildiğini söyler.

> **Hocanın vurgusu:** Bu sorularda görmediğiniz, bilmediğiniz bir şey yok. Derste gösterilen bilgileri yorumlamanız yeterlidir. Bir soruya hemen cevap veremiyorsanız demek ki o konu henüz tam oturmamıştır.

#### 2.1 Soru 1: En Az Sayıda Girdi İçin 1 Üreten İşlem

**Soru:** Aşağıdaki Boole işlemlerinden hangisi veya hangileri, toplamda en az sayıda ikili girdi için 1 değerini üretir? Seçenekler AND, OR, XOR, “hepsi” ve NOT’tur.

Hoca önce NOT’u eler, çünkü NOT ikili girdi almaz, tek girdi alır. Geriye AND, OR, XOR ve “hepsi” kalır. Üç işlemin doğruluk tablosunu yan yana yazmak yeterlidir:

| $a$ | $b$ | AND | OR | XOR |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 | 1 |

| $a$ | $b$ | AND | OR | XOR |
|---|---|---|---|---|
| 1 | 0 | 0 | 1 | 1 |
| 1 | 1 | 1 | 1 | 0 |
| **1 üreten çift sayısı** | | **1** | **3** | **2** |

AND yalnız bir girdi çifti için 1 ürettiğinden cevap AND'dir. Hoca bunu A şıkkı olarak belirtir.

### 2.2 Soru 2: Onaltılık Notasyonda İfade Edilemeyen Örüntü

**Soru:** Aşağıdaki bit örüntülerinden hangisi onaltılık (hexadecimal) notasyonda ifade edilemez? Şıklar: `11111111`, `1001`, `110011`, `100000000001`, `1111000000000000`.

Hoca, bir örüntünün onaltılık yazılabilmesi için ne gerektiğini sınıfa sorar.

**Soru:** Onaltılık notasyonda ifade edilebilmesi için ne olması gerekir? **Cevap:** Bir öğrenci bit sayısının 4'ün katı olması gerektiğini söyler, hoca da bunu onaylar.

Bir onaltılık basamak 4 biti kodlar. `110011` örüntüsünde 6 bit vardır. Sağdaki `0011` grubu kodlanır, ama soldaki `11` grubu tek başına 4 bit etmediği için kodlanamaz. Diğer şıkların bit sayıları 4'ün katıdır. Bu nedenle cevap C şıkkı, yani `110011` örüntüsüdür.

> **Ek bilgi (derste anlatılmadı):** Sayısal değer korunacaksa soldan iki sıfır eklenip `00110011` yazılarak bu değer `33` biçiminde onaltılık gösterilebilir. Ancak bu, örüntüyü 8 bite çıkarmak demektir. Sorunun "ifade edilemez" sözü büyük olasılıkla örüntünün hiç değiştirilmeden, 4 bitlik gruplara bölünmesini varsayar.

### 2.3 Soru 3: İkiye Tümleyen ile Toplama

**Soru:** İkiye tümleyen yaklaşımını kullanma şartıyla `00001111 + 10101010` işleminin sonucu nedir?

Hoca burada olağan ikili toplama yapıldığını söyler: 1'ler ve 0'lar toplanır, elde oluşunca bir sol basamağa aktarılır. Sağdan sola adımlar şöyledir:

1. $1 + 0 = 1$.
2. $1 + 1 = 2$. Basamağa 0 yazılır, elde 1.
3. $1 + 0 + 1 = 2$. Basamağa 0 yazılır, elde 1.
4. $1 + 1 + 1 = 3$. Basamağa 1 yazılır, elde 1.
5. $0 + 0 + 1 = 1$.
6. $0 + 1 = 1$.
7. $0 + 0 = 0$.
8. $0 + 1 = 1$.

```
  00001111
+ 10101010
----------
  10111001
```

Sonuç `10111001` olur. Onluk kontrol (ara adım eklendi): `00001111` sayısı 15, `10101010` sayısı ikiye tümleyende $-86$'dır. $15 + (-86) = -71$ ve $-71$ sayısının 8 bitlik ikiye tümleyen gösterimi `10111001` örüntüsüdür.

### 2.4 Soru 4: İkiye Tümleyen ile Çıkarma

**Soru:** Aynı yaklaşımla `00001111 - 10101010` işleminin sonucu nedir?

Hoca, çözüm yolunu sınıfa sorar ve iki farklı cevap alır.

**Soru:** Bu çıkarmada ne yapabiliriz? **Cevap:** Bir öğrenci alttaki sayıyı "eksiyle çarpmayı", yani sıfırları bire, birleri sıfıra çevirmeyi önerir. Başka bir öğrenci ikinci sayının ikiye tümleyenini alıp topladığını söyler. Hoca ikinci yolu onaylar.

Hocanın kurduğu düşünce şöyledir: Çıkarılan sayıyı eksi işaretiyle düşünürüz. Bu sayının karşılığını bulur, üstteki sayıyla toplarız. Yalnız bitleri ters çevirmek yetmez, sonuca 1 de eklenir:

1. `10101010` sayısının bitleri ters çevrilir: `01010101`.
2. Sonuca 1 eklenir: `01010110`. Bu sayı, çıkarılan sayının ikiye tümleyenidir.
3. Üstteki sayı olan `00001111` (15) ile toplanır.

Toplama sağdan sola şöyle ilerler: $1 + 0 = 1$; $1 + 1 = 2$, yani 0 elde 1; $1 + 1 + 1 = 3$, yani 1 elde 1; $1 + 0 + 1 = 2$, yani 0 elde 1; $0 + 1 + 1 = 2$, yani 0 elde 1; $0 + 0 + 1 = 1$; $0 + 1 = 1$; $0 + 0 = 0$.

```
  00001111
+ 01010110
----------
  01100101
```

Sonuç `01100101` olur. Hoca, sayıları onluğa çevirerek de hesap yapılabileceğini söyler. Onluk kontrol (ara adım eklendi): $15 - (-86) = 15 + 86 = 101$ ve $101 = 01100101_2$.

### 2.5 Soru 5: Hangi Sayı Negatif?

**Soru:** Aşağıdakilerden hangisi ikiye tümleyen notasyonuna göre negatif bir sayıyı temsil eder? Şıklar: `3F`, `55`, `A6`, `7E`, "hiçbiri".

Hoca, negatif sayının en soldaki bitinin 1 olması gerektiğini hatırlatır. Bu nedenle yalnız sayıların ilk onaltılık basamağına bakmak yeterlidir:

| İlk basamak | İkili karşılığı | En soldaki bit |
|---|---|---|
| 3 | 0011 | 0 |
| 5 | 0101 | 0 |
| A | 1010 | 1 |
| 7 | 0111 | 0 |

Yalnız `A` 1 ile başlar. Cevap C şıkkı, yani `A6`'dır. Sayının tamamını ikiliye çevirmeye gerek yoktur.

### 2.6 Soru 6: Bit Örüntüsünden Kesirli Sayıyı Bulmak

**Soru:** `01011100` örüntüsü bir kesirli sayıyı temsil eder. Dört bit mantis, üç bit üs (exponent), bir bit işaret biti olarak kullanılır. Bu örüntü aşağıdakilerden hangisidir? Şıklar: $-1\frac{1}{2}$, $1\frac{1}{2}$, $-\frac{3}{8}$, $\frac{3}{8}$, "hiçbiri".

Hoca "daha birinci saniyede hangi seçenekler elendi?" diye sorar ve bir öğrenci negatif seçeneklerin elendiğini söyler. Çünkü örüntünün ilk biti 0'dır, yani sayı pozitiftir.

| İşaret | Üs | Mantis |
|---|---|---|
| 0 | 101 | 1100 |

Geri kalan adımlar şöyledir:

1. Üs alanı `101`, yani 5'tir. 5'ten 4 çıkarılır: $5 - 4 = +1$. Bu, noktanın bir basamak sağa kayacağı anlamına gelir.
2. Mantis yazılır: `.1100`.
3. Nokta bir basamak sağa kaydırılır: $1.100_2$.
4. Noktanın solundaki 1, tam kısmı; sağındaki 1 ise $\frac{1}{2}$'yi temsil eder.

Sonuç $1\frac{1}{2}$'dir. Hoca sonucu "1 tam 1 bölü 2" olarak söyler.

**Soru:** $1.100_2$ kaça denk gelir? **Cevap:** Bir öğrenci 12 cevabını verir. Hoca bunun doğru olmadığını, arada nokta bulunduğunu, yani sayının kesirli olduğunu söyler. 12, noktayı yok sayıp `1100` örüntüsünü tam sayı gibi okumaktan çıkar.

### 2.7 Soru 7: Nümerik Değerleri En Az Verimle Kodlayan Yöntem

**Soru:** Aşağıdaki yöntemlerden hangisi nümerik değerlerin kodlanmasını en az verimlilikle gerçekleştirir? Şıklar: ikiye tümleyen notasyonu, excess notation, ASCII, floating point notation, işaretsiz tam sayı sistemi.

Hoca, bu sorunun öğrenilenleri yorumlayarak cevaplanması gerektiğini söyler.

**Soru:** Hangisidir? **Cevap:** Bir öğrenci ikiye tümleyen notasyonu der, hoca bunun doğru olmadığını söyler. Başka bir öğrenci ASCII der. Gerekçesi, tek bir sayıyı kodlamak için bile birden fazla byte gerekebilmesidir. Hoca bunu onaylar.

ASCII karakterleri kodlamak için kullanılır, diğer dört yöntem ise sayıları kodlamak için. ASCII'de her rakam ayrı bir karakterdir. Örneğin 1234 sayısında 1 için bir byte, 2 için bir byte, 3 için bir byte, 4 için bir byte kullanılır. Cevap ASCII'dir.

### 2.8 Soru 8: Run-Length Encoding ile Kazanç

**Soru:** `BBBBBBWBBBBWWWWBBBBW` dizisi run-length encoding ile kodlanırsa yüzde $X$ oranında daha az yer kaplaması beklenir. $X$ kaçtır?

Run-length encoding'de ardışık aynı karakterler sayılır ve "tekrar sayısı + karakter" biçiminde yazılır. Dizi şu gruplara ayrılır: 6 tane B, 1 tane W, 4 tane B, 4 tane W, 4 tane B, 1 tane W. Hoca kontrol için grupları toplar. Ara toplamlar 6, 7, 11, 15, 19 ve 20'dir, yani özgün dizi 20 karakterdir.

Kodlanmış hâl `6B1W4B4W4B1W` olur. Bu da 12 karakterdir. Kazanç oranı:

$$\frac{20 - 12}{20} = \frac{8}{20} = 0.40$$

Cevap $X = 40$'tır.

> **Ek bilgi (derste anlatılmadı):** Aynı örnekteki tek W'ler `1W` yazıldığında bir karakterlik veri iki karakter kaplar. Bu nedenle run-length encoding büyük olasılıkla ancak uzun tekrarlar içeren verilerde kazanç sağlar. Tekrarın az olduğu bir dizide kodlanmış hâl özgün diziden uzun çıkabilir.

### 2.9 Soru 9: 4096 Renk İçin Gereken Bit Sayısı

**Soru:** 4096 renk değerini saklamak için en az kaç bit gerekir?

Hoca "en kötü ihtimali" gösterir: 4096 sayısı 1'e inene kadar ardışık olarak 2'ye bölünür.

$$4096 \to 2048 \to 1024 \to 512 \to 256 \to 128 \to 64 \to 32 \to 16 \to 8 \to 4 \to 2 \to 1$$

Bu dizide 12 bölme vardır. Demek ki $2^{12} = 4096$ eder ve 4096 farklı rengi saklamak için en az 12 bit gerekir. Cevap 12'dir.

Hoca bir karşılaştırma da yapar: $2^8 = 256$ olduğundan ASCII tablosunda 256 farklı sembol 8 bitle, yani 1 byte ile saklanır.

> **Ders sonrası not:** Hoca ASCII tablosunda 256 sembol olduğunu söyledi. Büyük olasılıkla burada 8 bitlik genişletilmiş ASCII kastedildi. Standart ASCII 7 bitlik olup 128 sembol içerir. $2^8 = 256$ hesabı ise doğrudur.

Bir öğrenci, sınavda soruyu "her renk bir bittir, 4096 renk varsa 4096 bit gerekir" diye okuduğunu söyler. Oysa bir renk tek bir bite değil, bir bit örüntüsüne karşılık gelir. Burada sorulan, 4096 farklı örüntü üretebilmek için gereken bit sayısıdır.

### 2.10 Soru 10: XOR Kapılarından Oluşan Devre

**Soru:** Art arda bağlı üç kapıdan oluşan devrenin çıkışı nedir? İlk kapının girişleri 0 ve 1, sonraki kapıların ek girişleri sırasıyla 0 ve 1'dir.

Hoca, kapının simgesi hatırlanmıyorsa ders notlarından bakılabileceğini söyler ve simgenin hangi kapı olduğunu sorar. Cevap XOR'dur. XOR'da girdiler aynıysa 0, farklıysa 1 üretilir: $0 \oplus 0 = 0$, $0 \oplus 1 = 1$, $1 \oplus 0 = 1$, $1 \oplus 1 = 0$.

Devre soldan sağa izlenir. Her kapının çıkışı bir sonraki kapının girişi olur:

1. İlk kapı: $0 \oplus 1 = 1$.
2. İkinci kapı: $1 \oplus 0 = 1$.
3. Üçüncü kapı: $1 \oplus 1 = 0$.

Devrenin çıkışı 0'dır.

### 2.11 Soruların Ekrandan Tekrar Geçilmesi

Hoca tahtadaki çözümden sonra soruları bir kez de ekrandan hızlıca geçer. Cevaplar şöyle özetlenir:

| Soru | Konu | Cevap |
|---|---|---|
| 1 | En az girdi için 1 üreten işlem | AND |
| 2 | Onaltılık ifade edilemeyen örüntü | `110011` |
| 3 | `00001111 + 10101010` | `10111001` |
| 4 | `00001111 - 10101010` | `01100101` |
| 5 | Negatif sayı | `A6` |
| 6 | `01011100` kesirli sayısı | $1\frac{1}{2}$ |
| 7 | En az verimli kodlama | ASCII |
| 8 | Run-length encoding kazancı | %40 |
| 9 | 4096 renk için bit | 12 |
| 10 | XOR devresi | 0 |

Bu geçişte Soru 7 için ek bir örnek gelir: Hocanın verdiği sayı normal gösterimde 4 byte tutarken ASCII'de 6 byte kaplar, çünkü her rakam bir byte ile ifade edilir. Bu yüzden ASCII daha az verimlidir.

Soru 8 için de oran bir kez daha söylenir: Özgün dizi 20 byte, sıkıştırılmış bilgi ise "sayı ve kendisi" çiftleriyle toplam 12 byte'tır. Bu ikisi oranlandığında yer %40 azalmış olur.

## 3. Yazılı Soru 1: Onaltılık Karakter Dizisinin Onluk Karşılığı

### 3.1 Soru ve Öğrencilerin Yaklaşımları

**Soru:** Bir karakter dizisinde 0-9 rakamlarından ve A-F harflerinden oluşan $N$ adet karakter yer almaktadır. Onaltılık sayı sistemine göre verilmiş bu karakter grubunun onluk sayı sistemindeki karşılığını bulan algoritmanın akış diyagramını çizip örnekler için analizini yapınız. Örnekler: `1 0 A F` için `4271`, `C B A` için `3258`.

Hoca, çözüme geçmeden önce sınıftan yaklaşımlarını ister. Öğrenciler farklı yollar anlatır:

- **Önce harfleri sayıya çevirmek:** A, B, C, D, E, F karakterleri input alındıktan sonra sayıya çevrilir. Sonra her basamak en sağa uzaklığına göre 16, 16'nın karesi, 16'nın küpü ile çarpılır. Çarpan bir değişkende tutulup her adımda 16 ile güncellenir.
- **ASCII değerlerini kullanmak:** Bir öğrenci büyük A'nın 65'ten, sıfırın 48'den başladığını bildiğini söyler. A'nın 10'a denk gelmesi gerektiğinden, karakterden ayrıca 7 çıkarır. Çünkü $65 - 48 - 7 = 10$ olur.
- **Her harf için ayrı kontrol:** Bir öğrenci, ASCII değerleriyle ilerlenmemesi gerektiğini hatırladığı için her harf için ayrı `if` yazar: A ise 10, F ise 15 gibi. Sonucu doğru alır, ama bu yol uzundur.

- **Değer atayıp döngüyle toplamak:** A'dan F'ye kadar 10 ile 15 arasındaki değerler atanır. Çarpım değişkeni 1, sonuç değişkeni 0 yapılır. Döngüde değer çarpanla çarpılıp sonuca eklenir.

### 3.2 Hocanın Çözümü: Karakterin Sayısal Değeri

Hoca tahtaya bir karakter dizisi yazar: `char dizi[100]`. İçine sorudaki `1 0 A F` karakterleri konur.

Önce rakamlar ele alınır. `'0'` karakterinden `'0'` çıkarılırsa 0, `'1'` karakterinden `'0'` çıkarılırsa 1 elde edilir. Bu, `'9'` karakterine kadar sürer. Yani `dizi[i] - '0'` işlemi rakam karakterinin nümerik değerini verir.

Harfler için hoca şunu sorar: ASCII kodunu ezbere bilmiyorum diye düşünürsek ne yapmalıyım? Sınıftaki cevap adım adım kurulur:

1. `'A'` karakterinden `'A'` çıkarılırsa 0 elde edilir.
2. Oysa A'nın değeri 10 olmalıdır, bu yüzden 10 eklenir.
3. Aynısı `'F'` için de geçerlidir: `'F' - 'A' + 10 = 15`.

Böylece `dizi[i] - 'A' + 10` işlemi A-F aralığındaki karakterin değerini verir.

> **Hocanın vurgusu:** ASCII tablosundaki sayısal değerleri kullanmayın. Bunu derste defalarca söyledim. `'0'` ve `'A'` gibi karakter sabitleriyle çalışın.

**Soru:** Karakterden doğrudan 7 çıkarmanın bir sakıncası var mı? Başlangıç değerlerini biliyorum. **Cevap:** Hoca sakıncası olduğunu söyler. Sebebi derste net biçimde söylenen kuralın aksine ASCII değerlerinin kullanılmasıdır.

> **Ek bilgi (derste anlatılmadı):** Hoca bu kuralın gerekçesini burada açıklamadı. Gerekçe büyük olasılıkla, `'A'` yazımının 65'e göre daha okunur olması ve kodun belirli bir tablonun sayılarına bağlı kalmamasıdır.

### 3.3 Hangi Formülün Uygulanacağına Karar Vermek

Hoca, bu iki formülü kullanabilmek için tek bir ek işlem gerektiğini söyler. Bir öğrenci bunu bulur: Bir `if` ile gelen karakterin hangi aralıkta olduğu kontrol edilmelidir.

Girdilerin doğru olduğu varsayılabilir, kullanıcı hataları dikkate alınmaz. Bu yüzden tek kontrol yeterlidir. Karakter 0-9 aralığındaysa doğru kolda rakam formülü uygulanır. Değilse (`else`) karakter zaten A-F aralığındadır. Tersi de olur: Önce A-F kontrol edilir, `else` kolu 0-9 kabul edilir.

**Soru:** 0-9 aralığını iki karşılaştırmayla kontrol etmek yerine, doğrudan karakterin büyük A'dan küçük olup olmadığına bakabilir miyim? Gelen değerlerin doğru olduğunu varsaydım. **Cevap:** Olur, bir sorun çıkmaz.

### 3.4 Sayısal Değeri Basamak Değeriyle Çarpmak

Karakterin sayısal değeri bulunduktan sonraki iş, onu basamağının değeriyle çarpıp toplama eklemektir. Basamak değerleri sağdan sola $16^0, 16^1, 16^2, 16^3, \ldots$ şeklindedir. Hoca sorunun çözüm şeklinin bu olduğunu söyler. Akış diyagramını ve kodu derste yazmaz, öğrencilerin kendi çözmesini ister.

Aşağıdaki adımlar ve kod, hocanın anlattığı çözüm yolu ile öğrencilerin anlattığı çarpan güncelleme fikrinin birleştirilmesiyle hazırlanmıştır.

### 3.5 Akış Diyagramı (Numaralı Adımlarla)

1. Başla.
2. $N$ değerini ve `dizi` elemanlarını oku.
3. `sonuc = 0`, `carpan = 1` ata.
4. `i = N - 1` ata.
5. Karar: `i >= 0` mı? Hayırsa 11. adıma git.

6. Karar: `dizi[i]`, `'0'` ile `'9'` arasında mı? Evetse `deger = dizi[i] - '0'`, hayırsa `deger = dizi[i] - 'A' + 10`.
7. `sonuc = sonuc + deger * carpan`.
8. `carpan = carpan * 16`.
9. `i = i - 1`.
10. 5. adıma dön.
11. `sonuc` değerini yazdır.
12. Bitir.

### 3.6 Örnekler İçin Analiz

Soru, algoritmanın örnekler için analizini de ister. `1 0 A F` girdisi için izleme tablosu ($N = 4$):

| i | dizi[i] | deger | carpan | deger * carpan | sonuc |
|---|---|---|---|---|---|
| 3 | F | 15 | 1 | 15 | 15 |
| 2 | A | 10 | 16 | 160 | 175 |
| 1 | 0 | 0 | 256 | 0 | 175 |
| 0 | 1 | 1 | 4096 | 4096 | 4271 |

`C B A` girdisi için izleme tablosu ($N = 3$):

| i | dizi[i] | deger | carpan | deger * carpan | sonuc |
|---|---|---|---|---|---|
| 2 | A | 10 | 1 | 10 | 10 |
| 1 | B | 11 | 16 | 176 | 186 |
| 0 | C | 12 | 256 | 3072 | 3258 |

Çıktılar sırasıyla 4271 ve 3258'dir, sorudaki değerlerle aynıdır.

### 3.7 C Kodu

```c
/* Onaltılık karakter dizisinin onluk karşılığı */
#include <stdio.h>

int main()
{
    char dizi[100];
    int N, i, deger;
    int sonuc = 0, carpan = 1;

    scanf("%d", &N);
    for (i = 0; i < N; i++)
        scanf(" %c", &dizi[i]);

    for (i = N - 1; i >= 0; i--)   /* sağdan sola: 16^0, 16^1, ... */
    {
        if (dizi[i] >= '0' && dizi[i] <= '9')
            deger = dizi[i] - '0';
        else
            deger = dizi[i] - 'A' + 10;
        sonuc = sonuc + deger * carpan;
        carpan = carpan * 16;
    }
    printf("%d\n", sonuc);
    return 0;
}
```

Kod `gcc -Wall` ile uyarısız derlenmiş, `4 1 0 A F` girdisi için 4271, `3 C B A` girdisi için 3258 çıktısını vermiştir.

> **Ek bilgi (derste anlatılmadı):** `int` türünün 32 bit olduğu sistemlerde `sonuc` en fazla `7FFFFFFF` değerini tutabilir. Dizide 100 karakterlik yer olsa da daha büyük bir sayı `int`'e sığmaz. Bu nedenle uzun girdiler için büyük olasılıkla daha geniş bir tür gerekir.

### 3.8 Doğru Çözmek ve Verimli Çözmek

Bu sorudan sonra bir öğrencinin sorusu üzerine hoca, algoritma sorularına nasıl yaklaşılacağını anlatır. Yalnız programın çalışıp çalışmaması değil, çözüm şekli, olayın ele alınışı ve algoritmanın genel doğruluğu da önemlidir.

> **Hocanın vurgusu:** En doğrusunu çözmeye çalışın. Verimli çözmek iyidir, ama önce doğru çözmek önemlidir. Verimli çözmeye takılırsanız bazen doğru çözümü de üretemezsiniz.

Hoca öğrencilere bu soruyu yeniden çözmelerini önerir: akış diyagramını çizin, kodunu da yazın, sonra doğru çalışıp çalışmadığını kontrol edin.

## 4. Yazılı Soru 2: Matristen Sütun Silme

### 4.1 Soru ve Öğrencilerin Yaklaşımları

**Soru:** Verilen $N \times M$'lik bir matrisin $X$ numaralı sütununun silinmesini sağlayan algoritmanın akış diyagramını çizip kodunu C dilinde yazınız. Sütun silme işlemi sırasında hücreler sola kaydırılmalıdır.

Hoca sınıfa nasıl yapılması gerektiğini sorar. İlk cevaplar karışıktır:

- Bir öğrenci `for` döngüleriyle indislerden birini sabit tutup diğerini artırmayı anlatır. Hoca düşüncenin doğru olabileceğini, ama anlatımın karıştığını söyler.
- Bir öğrenci önce `while` açıp içinde `for` ile kontrol etmeyi önerir. Hoca burada `while` kullanmayı gerektiren şeyin ne olduğunu sorar ve bu şekilde düşünülmediğini söyler.
- Bir öğrenci iki `for` döngüsü kullandığını anlatır: İkinci sütunu üçüncüye eşitlemiş, üçüncüyü dördüncüye eşitlemiş, en sonda dördüncüyü sıfır yapmıştır. Hoca bu öğrenciyle adımların üzerinden geçer ve yaklaşımın genel olarak doğru olduğunu söyler.

### 4.2 Hocanın Çözümü

Hoca tahtaya örneği yazar, $X = 2$ verilmiştir:

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix}$$

Silinecek olan ikinci sütundur, yani 2, 6, 10 değerleri. Algoritma şöyledir:

1. Silinmek istenen sütunun bir yanındaki sütundan başlanır.
2. O sütunun bütün satırlarındaki elemanlar bir sol sütuna kaydırılır.
3. Bir sonraki sütuna geçilir ve aynı işlem yapılır.
4. Son sütuna kadar böyle devam edilir.
5. En sonda boşalan son sütuna sıfır yazılır.

Örnekte adımlar şöyle ilerler:

- **Üçüncü sütun ikinciye kopyalanır:** 2, 6, 10 değerleri yerini 3, 7, 11 değerlerine bırakır.
- **Dördüncü sütun üçüncüye kopyalanır:** Üçüncü sütun 4, 8, 12 olur.
- **Son sütun sıfırlanır:** Dördüncü sütun 0, 0, 0 olur.

$$\begin{bmatrix} 1 & 3 & 4 & 0 \\ 5 & 7 & 8 & 0 \\ 9 & 11 & 12 & 0 \end{bmatrix}$$

### 4.3 Sıfırlamayı Nerede Yapmalı?

Öğrencilerin sorularıyla sıfırlama adımı ayrıntılandırılır.

**Soru:** Ben sıfırlamayı döngünün içinde her seferinde yapıyordum. Sonuçta en son sütun yine sıfır oluyor, sonuç yanlış çıkmıyor. Bu sorun mu? **Cevap:** Hoca, öğrencinin bunu fark etmesini takdir eder. Verimli çözüm, sıfırlamayı bir defa yapmaktır. Tekrar tekrar yapmak bir hatadır, ama çok büyük bir hata değildir.

**Soru:** Tekrar tekrar sıfırlama ne demek? **Cevap:** Sıfırlamayı böyle yapan öğrenci açıklar. Üçüncü sütun ikinciye atanınca 2, 6, 10 olan kısım 3, 7, 11 olur. Sonra üçüncü sütun 0, 0, 0 yapılır. Bir sonraki adımda 4, 8, 12 bu sıfırların üzerine yazılır. Böylece sonuç doğru çıkar ama her adımda fazladan iş yapılmış olur.

**Soru:** Son sütunu sıfırlamak için döngünün içinde sütun numarası son sütuna eşit mi diye kontrol ettim. Bu fazladan bir işlem mi? **Cevap:** Evet, böyle bir kontrolü gerektiren bir durum yoktur. Hangi sütunun boşaldığı bellidir, doğrudan o sütunun satırları sıfırlanır. Hoca bunu da güzel bir tespit olarak niteler.

Bir öğrenci de yalnız sıfırlama kısmını yapıp sola kaydırmayı atladığını söyler. Hoca bunun sorunun çok önemli bir kısmının yapılmamış olması demek olduğunu belirtir. Kaydırma olmadan sütun silinmiş sayılmaz.

### 4.4 Döngü Seçimi: for mu while mı?

**Soru:** İki `for` yerine bir `for` bir `while` kullandım. Bu da çalışıyor, sorun olur mu? **Cevap:** Adım sayısı belli olan işlerde `for`, belli olmayan işlerde `while` kullanılmalıdır. Burada adım sayısı bellidir, bu yüzden `for` gerekir. `while` kullanmak teknik olarak yanlış oluşturmaz, program yine çalışır. Ama kavramsal bir sıkıntı yaratır.

> **Hocanın vurgusu:** `for` gereken yerde `while`, `while` gereken yerde `for` kullanmak kavramsal bir hatadır. Bundan sonraki çalışmalarınızda bu ayrıma dikkat edin.

### 4.5 Son Sütunu Hiç Okumamak

**Soru:** Algoritmayla ilgili bir önerim var: Son sütunu sıfıra eşitlemek yerine, matrisi okuma kısmında son sütunu okumadan da yapılabilir mi? **Cevap:** Tabii ki olur. Buradaki amaç, sorunun gerektirdiği biçimde bir problem kurgulamaktı.

Hoca, gerçek hayatta boşalan kısmın bellekte yer kaplamayacak şekilde düzenleneceğini ve bunun başka sürümlerinin ileride görüleceğini ekler.

### 4.6 Akış Diyagramı ile C Kodunda İndisler

**Soru:** Dizilerin indisi C'de sıfırdan başlıyor, ama akış diyagramında birden başlatıyoruz. Bunları eşitlememiz gerekir mi? **Cevap:** İkisi de kendi içinde tutarlıysa sorun yoktur. İkisini aynı da kullanabilirsiniz, biri 1, biri 0 olacak şekilde ayrı ayrı da. Yeter ki doğru çalışsın.

Hoca bu sorunun da kodunu ve akış diyagramını derste yazmaz, öğrencilere bırakır. Aşağıdaki adımlar ve kod, hocanın anlattığı çözüm yolundan hazırlanmıştır. Akış diyagramında indisler 1'den, C kodunda 0'dan başlar. Yukarıdaki cevapta söylendiği gibi ikisi de kendi içinde tutarlıdır.

### 4.7 Akış Diyagramı (Numaralı Adımlarla)

İndisler 1'den başlar:

1. Başla.
2. $N$, $M$, $X$ değerlerini ve matrisi oku.
3. `j = X + 1` ata.
4. Karar: `j <= M` mi? Hayırsa 10. adıma git.
5. `i = 1` ata.
6. Karar: `i <= N` mi? Hayırsa 9. adıma git.
7. `A[i][j-1] = A[i][j]`, `i = i + 1`.

8. 6. adıma dön.
9. `j = j + 1` ata ve 4. adıma dön.
10. `i = 1` ata.
11. Karar: `i <= N` mi? Hayırsa 14. adıma git.
12. `A[i][M] = 0`, `i = i + 1`.
13. 11. adıma dön.
14. Matrisi yazdır.
15. Bitir.

### 4.8 C Kodu

```c
/* N x M matristen X numaralı sütunu silme */
#include <stdio.h>

int main()
{
    int N, M, X, i, j;
    int matris[20][20];

    scanf("%d %d %d", &N, &M, &X);
    for (i = 0; i < N; i++)
        for (j = 0; j < M; j++)
            scanf("%d", &matris[i][j]);

    for (j = X; j < M; j++)        /* X. sütun (1'den sayılınca) indis X-1'dir */
        for (i = 0; i < N; i++)
            matris[i][j - 1] = matris[i][j];

    for (i = 0; i < N; i++)        /* boşalan son sütun bir kez sıfırlanır */
        matris[i][M - 1] = 0;

    for (i = 0; i < N; i++)
    {
        for (j = 0; j < M; j++)
            printf("%d ", matris[i][j]);
        printf("\n");
    }
    return 0;
}
```

Kod `gcc -Wall` ile uyarısız derlenmiş, dersteki matris ve $X = 2$ girdisi için yukarıdaki sonuç matrisini vermiştir.

## Dersin Toparlaması

Bu derste birinci ara sınavın test ve yazılı soruları çözüldü. Test bölümünde mantık kapılarının doğruluk tabloları, onaltılık notasyon, ikiye tümleyen ile toplama ve çıkarma, kayan noktalı gösterimin çözülmesi, ASCII'nin sayıları kodlamadaki verimsizliği, run-length encoding ve bit sayısı hesabı tekrar edildi. Yazılı bölümde ise iki algoritma problemi ele alındı: Karakter sabitleriyle onaltılık dizinin onluk karşılığını bulmak ve bir matristen sütun silip kalan hücreleri sola kaydırmak. İki problemde de asıl mesaj, ne istendiğini anlamak, önce doğru çözmek, sonra gereksiz işlemleri atmak ve adım sayısına uygun döngüyü seçmekti.

Hoca, sınavda çözülemese bile bu soruları oturup sıfırdan yeniden çözmenin bir yatırım olduğunu söyler. Sonraki derste matrislerle devam edileceğini, birkaç yeni bilgi verileceğini ve yeni algoritmik problemlere geçileceğini belirtir.

**Not:** İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.